병원 배송 로봇 SW 아키텍처 — 지금 구현 기준 (2026-10-21)

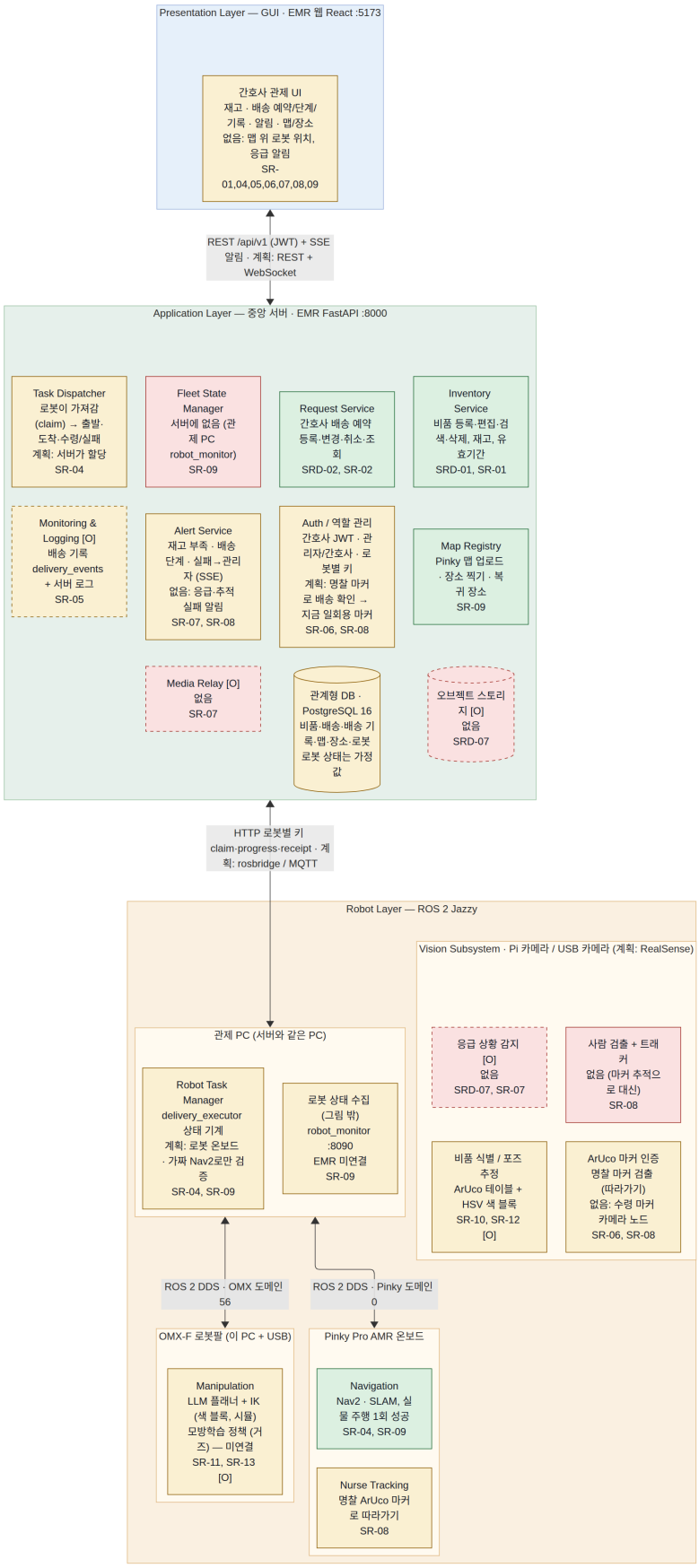
- 대상: 팀원 전체(웹·로봇·OMX). 팀 **Lv.1 SW 블록도**(3계층)를 지금 `~/dev_ws` 코드에 맞춰 다시 그리고, 블록마다 실제 파일·클래스를 붙였다.
- 기준: `main` (2bb8227) + 작업 중 변경(EMR 배송 기록 — 로드맵 4단계, 아직 커밋 전).
- **이 문서에서 뺀 것**(사용자 결정 2026-10-21): 환자 UI, 환자 요청·역할별 품목 분기, 비상 조작 콘솔, Safety Monitor.
- 파일: 이 문서 · 그림 `sw_3layer.png` · draw.io용 Mermaid 원본 `sw_3layer.mmd` (draw.io 메뉴 배치 → 삽입 → 고급 → Mermaid에 붙여 넣기).
- 경로 기준: 백엔드 `emr/backend/app/`, 프론트 `emr/frontend/src/`, 로봇 `robot/src/`.

1. 한눈에 보기

상태	개수	블록
구현	4	Request Service, Inventory Service, Map Registry, Navigation
일부·다르게	11	간호사 관제 UI, Task Dispatcher, Alert Service, Auth/역할, Monitoring & Logging [O], 관계형 DB, Robot Task Manager, Nurse Tracking, Manipulation, 비품 식별/포즈, ArUco 마커 인증
없음	5	Fleet State Manager(서버), Media Relay [O], 오브젝트 스토리지 [O], 응급 상황 감지 [O], 사람 검출 + 트래커

필수 블록 중 비어 있는 것: **Fleet State Manager**(맵 위 로봇 위치 — EMR 3단계), **수령 마커 카메라 노드**(ArUco 마커 인증의 빈 부분 — 배송 종단 시험에 필요), **사람 검출 YOLO**(지금은 마커 추적으로 대신).

2. 3계층 구조



색: 초록 = 구현 · 노랑 = 일부·다르게 · 빨강 = 없음 · 점선 테두리 = Priority O(옵션). 연결선에는 지금 통신 방식과 원래 계획을 함께 적었다.

계층	어디서 도나
Presentation	EMR 웹(React 19 + Vite, `:5173`). 간호사·관리자가 같은 화면을 쓰고 역할로 메뉴를 나눈다
Application	EMR API(FastAPI + SQLAlchemy 2, `:8000`) + PostgreSQL 16(`:5432`). Docker로 상시 기동
Robot	**관제 PC**(서버와 같은 PC: 배송 실행기·로봇 상태 수집·OMX 노드) + **Pinky Pro 온보드**(Nav2·따라가기·Pi 카메라). ROS 2 Jazzy

3. 계획과 구조가 다른 곳

#	계획(Lv.1 블록도)	지금	정할 것
1	GUI ↔ 서버 REST + WebSocket	REST + SSE(알림만 서버 → 브라우저)	그림을 SSE로 고칠지
2	서버 ↔ 로봇 rosbridge / MQTT	HTTP(로봇별 `X-Robot-Api-Key`) + 관제 PC에서 ROS 2 DDS(Pinky 도메인 0, OMX 56)	그림을 지금 방식으로 고칠지
3	Robot Task Manager가 로봇 온보드	관제 PC `delivery_executor`	Pinky upstream을 고치지 않으려고 관제 PC에 둠
4	Task Dispatcher가 로봇에 할당	로봇이 3초마다 가져감(claim, `FOR UPDATE SKIP LOCKED`)	10-19 결정 유지 여부
5	Fleet State Manager가 서버 안	관제 PC `robot_monitor`(`:8090`), EMR과 연결 안 됨	EMR 3단계에서 서버로 보고
6	간호사 명찰 마커로 배송 확인	일회용 수령 마커(`DICT_4X4_250` 50–249)로만 수령, 명찰(0–49)은 따라가기 전용	10-20 결정 유지 여부
7	RealSense RGB-D	Pinky Pi 카메라, OMX USB 카메라(깊이 없음)	장비 계획

4. 기능 묶음별 파일·클래스

블록도의 블록을 기준으로 묶었다. 이름이 정해지지 않은 묶음은 코드 모듈 이름을 따서 중간 명칭을 붙였다. 표기: `<파일, 클래스·함수>` : 설명.

Application Layer (중앙 서버 · EMR)

4.1 배송 예약 `delivery` — Request Service · 구현

- `<api/v1/deliveries.py, router>` : 배송 예약 등록·목록·한 건·변경·취소, 한 건의 기록(`/{id}/events`)
- `<services/delivery.py, DeliveryService>` : 예약 규칙(미래 시각, 장소·비품 확인, 본인만, reserved일 때만 변경·취소), 재고 차감·복구
- `<repositories/delivery.py, DeliveryRepository>` : 배송 조회(품목·목적지 함께 읽기, 잠금), 본인 목록, 품목 교체
- `<models/delivery.py, Delivery · DeliveryItem>` : 배송 헤더(누가·어디로·언제·상태)와 품목 줄
- `<schemas/delivery.py, DeliveryCreate · DeliveryUpdate · DeliveryOut>` : 예약 요청·응답 모양
- `<pages/Deliveries.tsx, Deliveries>` : 배송 예약 화면(목록·변경·취소·기록·수령 마커 버튼)
- `<features/deliveries/DeliveryForm.tsx, DeliveryForm>` : 예약 등록·변경 입력 폼
- `<features/deliveries/DeliveryProgress.tsx, DeliveryProgress>` : 행마다 단계 표시(예약 → 출발 → 도착 → 수령/실패)

4.2 비품 재고 `supply` — Inventory Service · 구현

- `<api/v1/supplies.py, router>` : 비품 검색·한 건·등록·수정·삭제 표시

- `<services/supply.py, SupplyService · auto_expiry_date>` : 비품 규칙, 유효기간 자동(6개월~1년)
- `<repositories/supply.py, SupplyRepository>` : 비품 조회, 여러 행 잠금(재고 차감 때)
- `<models/supply.py, Supply>` : 비품 테이블(재고 = 예약 가능 수량, 부족 기준)
- `<schemas/supply.py, SupplyCreate · SupplyUpdate · SupplyOut>` : 비품 요청·응답 모양
- `<pages/Supplies.tsx, Supplies>` : 비품 화면

4.3 배송 배정·진행 `dispatch` — Task Dispatcher · 다르게(로봇이 가져감)

- `<api/v1/robot_access.py, router>` : 로봇 전용 API — `/robot/me`, claim, progress, receipt(로봇 키 인증)
- `<services/dispatch.py, DispatchService>` : 배송 가져가기(claim), 도착·실패 보고, 일회용 마커로 수령 확인
- `<services/dispatch.py, RobotOverview>` : `/robot/me` 내용(로봇·복귀 장소·맡은 배송)
- `<repositories/delivery.py, DeliveryRepository.lock_next_claimable>` : 맡을 배송 한 건 잠그기(`FOR UPDATE SKIP LOCKED`)
- `<schemas/delivery.py, RobotDeliveryOut · DeliveryProgressIn · DeliveryReceiptIn · RobotMeOut>` : 로봇용 응답(마커·간호사 정보 없음)·보고 모양
- `<emr/backend/scripts/simulate_robot.py>` : 로봇 흉내 도구(실물 없이 시험)
- 계약: ../api_contract_delivery_2026-10-20.md

4.4 실시간 알림 `notification` — Alert Service · 일부

- `<core/notification_hub.py, NotificationHub>` : DB LISTEN → 접속한 간호사에게 나눠 줌(본인·역할별로 거름)
- `<repositories/notification.py, NotificationRepository>` : 트랜잭션 안에서 `pg_notify` 발행(commit될 때만 전달)
- `<services/low_stock.py, LowStockNotifier · is_low_stock>` : 재고 부족·해소 알림
- `<services/delivery_notifier.py, DeliveryNotifier>` : 배송 출발·도착·수령·실패 알림(실패는 관리자에게도)
- `<api/v1/notifications.py, router>` : 지금 부족한 비품 목록(`/low-stock`), SSE 스트림(`/stream`)
- `<features/notifications/useNotificationStream.ts, useNotificationStream>` : SSE 연결·재연결
- `<features/notifications/NotificationBell.tsx, NotificationBell>` : 상단 알림 종(재고 부족)
- `<features/notifications/Toasts.tsx · toastStore.ts, Toasts>` : 배송 단계 토스트(파랑·초록·빨강)
- 없음: 응급 알림, 추적 실패 알림

4.5 인증·권한 `auth` — Auth / 역할 관리 · 일부

- `<api/v1/auth.py, router>` : 로그인, 내 정보, 비밀번호 변경
- `<services/auth.py, AuthService>` : 사번·비밀번호 확인, 토큰 발급
- `<core/security.py, hash_password · create_access_token · generate_robot_api_key>` : bcrypt, JWT, 로봇 키 생성·해시
- `<core/deps.py, get_current_nurse · get_current_admin · get_current_robot>` : 요청마다 간호사·관리자·로봇 확인
- `<api/v1/nurses.py, router>` · `<services/nurse.py, NurseService>` · `<repositories/nurse.py, NurseRepository>` : 간호사 관리(등록·수정·비활성화·비밀번호 초기화·명찰 마커 번호)
- `<models/nurse.py, Nurse>` : 간호사 테이블(역할, 명찰 마커 0–49)
- `<schemas/auth.py, LoginIn · TokenOut · PasswordChange>` · `<schemas/nurse.py, NurseCreate · NurseUpdate · NurseOut>` : 요청·응답 모양
- `<pages/Login.tsx · Password.tsx · Nurses.tsx>` : 로그인, 비밀번호 변경, 간호사 관리 화면

- `<features/markers/arucoDictionary.ts · ArucoMarker.tsx · MarkerPrintDialog.tsx>` : ArUco 비트 표, 마커 그리기, 명찰 인쇄
- 계획과 다름: 명찰 마커로 배송을 확인하지 않는다(수령은 일회용 마커)

4.6 배송 기록 `delivery_events` — Monitoring & Logging [O] · 일부

- `<api/v1/delivery_events.py, router>` : 관리자 배송 기록 목록(거르기·최근 순)
- `<services/delivery_events.py, DeliveryEventRecorder>` : 예약·변경·취소·출발·도착·수령·실패를 같은 트랜잭션에 기록
- `<services/delivery_events.py, DeliverySnapshot>` : 변경 전후 비교용 값(이름은 당시 값)
- `<services/delivery_events.py, DeliveryEventService>` : 한 건 기록(본인·관리자), 관리자 목록
- `<repositories/delivery_event.py, DeliveryEventRepository>` : 기록 조회(한 배송 / 거르기)
- `<models/delivery_event.py, DeliveryEvent>` : 기록 테이블(JSONB 내용, 로봇 이름 스냅숏)
- `<schemas/delivery_event.py, DeliveryEventOut · AdminDeliveryEventOut>` : 기록 응답
- `<pages/DeliveryLog.tsx, DeliveryLog>` : 관리자 "배송 기록" 화면
- `<features/deliveries/DeliveryHistoryDialog.tsx · deliveryEventText.ts>` : 간호사 "기록" 창, 기록 문장 만들기
- `<core/logging.py, configure_logging>` : 서버 로그(structlog JSON)
- 없음: 시스템 상태 로그

4.7 맵·장소 `map` · `location` — Map Registry · 구현

- `<api/v1/maps.py, router>` : 맵 등록·수정·삭제, 파일 업로드·교체, 이미지 내려받기
- `<services/map.py, MapService · MapFile>` : 맵 규칙(이름·파일 중복, 쓰는 장소가 있으면 삭제 막기)
- `<services/map_files.py, parse_map_yaml · to_png · MapYamlInfo>` : Pinky 맵 yaml 읽기, PGM → PNG 변환
- `<repositories/map.py, MapRepository>` : 맵 조회(이름·파일 이름으로, 쓰는 장소가 있는지)
- `<api/v1/locations.py, router>` · `<services/location.py, LocationService>` · `<repositories/location.py, LocationRepository>` : 장소(좌표·방향) 관리, 복귀 장소인 장소는 삭제 막기
- `<models/map.py, Map>` · `<models/location.py, Location>` : 맵(resolution·origin·이미지)·장소 테이블
- `<schemas/map.py, MapCreate · MapOut>` · `<schemas/location.py, LocationCreate · LocationOut>` : 요청·응답 모양
- `<pages/MapView.tsx, MapView>` : 맵 화면
- `<features/maps/LocationPicker.tsx · mapGeometry.ts>` : 맵을 눌러 장소 찍기, 미터 ↔ 픽셀 변환

4.8 로봇 관리 `robot` (중간 명칭: Robot Registry) — Fleet State Manager의 바탕 · 서버 집계는 없음

- `<api/v1/robots.py, router>` : 로봇 등록·수정·삭제, API 키 발급·폐기
- `<services/robot.py, RobotService>` : 로봇 규칙(종류 변경 불가, 진행 중 배송이 있으면 삭제 막기, 복귀 장소 지정)
- `<repositories/robot.py, RobotRepository>` : 로봇 조회(이름·키 해시로, 잠금)
- `<models/robot.py, Robot>` : 로봇 테이블(종류, **상태는 손으로 넣는 가정값**, 키 해시, 복귀 장소)
- `<schemas/robot.py, RobotCreate · RobotUpdate · RobotOut · RobotApiKeyOut>` : 요청·응답 모양
- `<pages/Sites.tsx, Sites>` · `<features/robots/RobotApiKeyDialog.tsx>` : 장소·맵·로봇 탭, 키를 한 번만 보여 주는 창
- 없음: 로봇 위치·상태 보고 받기, 집계, 충돌 위험 감지(EMR 로드맵 3단계)

4.9 공통 기반 `core` (중간 명칭: Platform) — 블록도 밖

- `<core/config.py, Settings>` : 환경 설정(`.env` , pydantic-settings)
- `<core/database.py, get_db>` : DB 연결·세션(응답 전에 commit)
- `<core/exceptions.py, DomainError · NotFound · Conflict …>` : 도메인 오류 → `{"code","detail"}` 응답
- `<models/base.py, Base · TimestampMixin>` · `<repositories/base.py, BaseRepository>` : 모델·리포지토리 공통 뼈대(`get` · `list` · `add`)
- `<models/enums.py, DeliveryStatus · DeliveryEventType · RobotType …>` : 상태·종류 값 목록(CHECK 제약용)
- `<schemas/common.py, OrmOut>` : 모든 응답 스키마의 베이스
- `<api/client.ts · hooks.ts · types.ts · errorMessages.ts>` : 프론트 API 호출·Query 훅·타입·오류 안내
- `<components/Layout.tsx · ui.tsx · ErrorDialog.tsx>` · `<router.tsx · main.tsx>` : 공통 틀·UI 조각·오류 팝업·경로

Robot Layer (ROS 2)

4.10 배송 실행기 `delivery_executor` — Robot Task Manager · 다르게(관제 PC)

- `<fms/delivery_executor/logic.py, DeliveryWorker>` : 상태 기계(가져가기 → 주행 → 도착 → 마커 대기 → 수령/실패 → 복귀)
- `<fms/delivery_executor/logic.py, WorkerSettings · WorkerState · compare_maps>` : 설정, 진행 상태, 로봇 맵과 EMR 맵 비교
- `<fms/delivery_executor/emr_client.py, RobotEmrClient>` : EMR 로봇 API 호출(로봇 키)
- `<fms/delivery_executor/ros_io.py, RobotRos · RosNavigator · RosMarkers>` : 로봇별 ROS 연결, Nav2 `navigate_to_pose` , `receipt_marker` 토픽 받기
- `<fms/delivery_executor/main.py>` : 시작, 로봇별 워커 실행, 종료 신호 처리
- 가짜 Nav2(도메인 77)로만 검증. 실물은 비품실 모방학습이 끝난 뒤

4.11 로봇 상태 수집 `robot_monitor` — 블록도 밖 · 일부(관제 PC)

- `<fms/robot_monitor/monitor_node.py, RobotMonitor>` : 로봇 위치(TF)·배터리를 읽어 `:8090` HTTP/SSE로 제공
- `<fms/robot_monitor/state.py, RobotState>` : 로봇 한 대의 상태 값
- EMR로 보내지 않고, 관제 화면도 없다

4.12 주행 `navigation` — Navigation · 구현

- Pinky upstream Nav2·SLAM(`pinky_pro` , 저장소에 넣지 않음 — 루트 README 부록 A로 받음)
- `<fms/test_waypoint/test_waypoint.py, PinkyWebClient · Waypoint>` : EMR 장소 좌표로 Pinky 이동 시험(시험용)
- `<fms/test_waypoint/emr_map_export.py>` : EMR 맵을 로봇용 파일로 내보내기

4.13 간호사 따라가기 `pinky_marker` — Nurse Tracking · 일부(Pinky 온보드)

- `<pinky_onboard/pinky_marker/follower_node.py, NurseFollower>` : 명찰 마커를 보고 `/cmd_vel` 발행, Nav2 주행 중이면 멈춤, 켜기·끄기 서비스
- `<pinky_onboard/pinky_marker/follow_logic.py, FollowParams · decide_state · compute_command>` : 따라가기 상태·속도 계산(ROS 없이 시험 가능)
- `<pinky_onboard/pinky_marker/camera_node.py, CameraPublisher>` : Pi 카메라 → `/camera/image_raw`

- 가짜 카메라로만 검증

4.14 로봇팔 조작 `vla_planner_robot` — Manipulation · 일부(OMX)

- `<vla_planner_robot/vla_planner/planner_node.py, PlannerNode>` : 명령 문장 → LLM 계획 → 단계별 실행 요청 → 카메라로 확인
- `<vla_planner_robot/vla_planner/llm_plan.py, PlanError · build_schema · resolve_plan>` : Ollama(gemma4:e2b) 호출, JSON 계획을 좌표로 바꾸기
- `<vla_planner_robot/vla_executor/executor_node.py, PickPlaceExecutor · Arm>` : MoveIt으로 집어 옮기기, 두 팔 운반
- `<vla_planner_robot/vla_executor/omx_ik.py, top_down_ik · fk_tool>` : OMX-F 위에서 집는 IK
- 팀원 모방학습 정책(`il` 브랜치, LeRobot SmolVLA, 거즈): 아직 연결 안 됨 — 설계 초안은 `calibration` 브랜치의 `docs/omx_skill_interface_2026-10-21.md`

4.15 비전 `vision` — Vision Subsystem · 일부

- `<vla_planner_robot/vla_perception/perception_node.py, PerceptionNode>` : 위쪽 카메라 → `/vla/blocks`(색 블록 위치) — 비품 식별·포즈
- `<vla_planner_robot/vla_perception/table_geometry.py, TableHomography · Undistorter>` : ArUco 4개로 테이블 좌표, 렌즈 왜곡 보정
- `<vla_planner_robot/vla_executor/grasp_check.py, grasp_fill>` : 손목 카메라로 잡았는지 확인
- `<pinky_onboard/pinky_marker/vision.py, make_detector · marker_geometry>` : ArUco 검출(명찰 0–49, 수령 마커 50–249를 같은 사전으로)
- 없음: 수령 마커 카메라 노드, 사람 검출 YOLO, 응급 감지

5. 관련 문서

- `../emr_status_2026-10-21.md` — EMR 웹 진행 현황(로드맵 단계)
- `../api_contract_delivery_2026-10-20.md` — EMR ↔ 배송 실행기 API 계약
- `../pinky_multi_robot_2026-10-20.md` — 관제 PC 구조와 여러 대 운영(로봇마다 ROS 도메인)
- `docs/omx_skill_interface_2026-10-21.md`(`calibration` 브랜치, 아직 `main`에 없음) — OMX 스킬 인터페이스 설계(LLM 플래너 + IK + 모방학습)